React

Frontend Web Frameworks

IT MSc Guest Lecture

5th March 2026

Who am I? What are my credentials?

- I'm Jake - PhD student working in programming languages
- Part-time professional web developer
    - Most recently, on some critical school infrastructure
- Lots of web app side projects
- Winner of many hackathons

```
I've written a lot of react, and been paid for it
```

Why do we need a frontend framework?

- HTML/CSS/JS roughly correspond to 'structure, style and logic'
- We deal with each independently

→ this turns out to be an improper separation of concerns

- It's a **vertical** slicing of the application

→ which makes it hard to build **composable abstractions**

→ Structure and logic benefit from being more tightly coupled

Frontend Frameworks

Modern frontend web frameworks attempt to rectify this by undoing the separation (to some degree)

I'm going to talk react, but the broad strokes all basically apply to any of the modern web frameworks

Some argument to say that even the structure/style separation is not great

See: **tailwind**

```
big idea: Make inline CSS feasible and comfortable
```

We won't discuss that further today however

Components

React consolidates logic and structure into a unit of encapsulation called a *component*

Components are reusable blocks of code that represent one (usually visual) element on the screen

Components

More than just a templating system

→ The logic related to the element is *also* encapsulated

```
More than just re-use - the encapsulation also helps
structure code more sensibly.
```

- Component abstractions are naturally composable - unlike raw html/javascript

React in three ideas:

1. function components
2. props
3. state & effects (hooks)

Function components

In react a *component* is just a special kind of function

- Component: a function which returns HTML

Example:

```
function NameCard() {
  return <p>Hello Jake!</p>
}
```

This syntax is called "JSX"
"JavaScript with Xml"

You can't run this directly in the browser - you need to pre-process it

Components can call other components:

```
function NameCard() {
  return <p>Hello Jake!</p>
}


function Page() {
  return <div>
    <NameCard/>
  </div>
}
```

This is what makes components a *composable* abstraction - you can build components out of other components

We can insert values from javascript dynamically:

```
function NameCard() {
  let name = "Jake"

  return <p>Hello {name}!</p>
}
```

A very common pattern: iterating over a list:

```
function NameCard() {
  let names = ["Jake", "Nikela", "Jeremy"]

  return <div className="flex flex-col gap-2" >
    {names.map(name =>
      <p>Hello {name}!</p>
    )}
  </div>
}
```

Note: we use `className` not `class` (since `class` is a JS keyword)

Props

If components are just functions can they take arguments?

- Yes they can!
- We call these arguments *props* - short for *properties*
- However, there's a special convention for setting these up.

All the props get passed in as fields on a single object

```
function NameCard(props) {
  return <p>Hello {props.name}!</p>
}
```

We can use javascript's *destructuring assignment* to make this a bit cleaner:

```
function NameCard({name}) {
  return <p>Hello {name}!</p>
}
```

At the call site, props just look like attributes:

```
function NameCard({name}) {
  return <p>Hello {name}!</p>
}

function Page() {
  return <div>
    <NameCard name="Jake"/>
  </div>
}
```

Again, we can use `{}` to do things dynamically

```
function NameCard({name}) {
  return <p>Hello {name}!</p>
}

function Page() {
  let names = ["Jake", "Nikela", "Jeremy"]

  return <div className="flex flex-col gap-2" >
    {names.map(name =>
      <NameCard name={name} />
    )}
  </div>
}
```

Hooks

- So far, I've shown you a nice way to handle templating
  - Not so different to django
- React's real power is that it also lets you program interactive things
- The tools we use to do this are hooks
- Today, we will look at the two most important ones:
  - `useState`
  - `useEffect`

State

- In react, "state" just refers to some data that can change
- We need to handle this carefully, because whenever state changes, we need to update what we put on the screen
- We call this "re-rendering"; it happens at the component level
- In order for react to know that a component needs to be re-rendered, we need to handle state in a special way that it understands

Anatomy of a `useState` call

```
import { useState } from 'react'

const [state, setState] = useState(init)
//                        ^^^^^^^^
```

- To create a peice of state, we call the `useState` hook
- A hook is a special function which "hooks into" the component lifecycle
- In this case, it helps determines when to re-render a component

Anatomy of a `useState` call

```
const [state, setState] = useState(init)
//                                ^^^^
```

- When we call `useState`, we can pass in an argument
- This is the *initial value* of the state

Anatomy of a `useState` call

```
const [state, setState] = useState(init)
//    ^^^^^^^^^^^^^^^^^^
```

- This returns an array with two values in it
- By convention, we always destructure these two out

Anatomy of a `useState` call

```
const [state, setState] = useState(init)
//     ^^^^^
```

- The first element of the array is the actual state
- This variable contains the current value of our state

Anatomy of a `useState` call

```
const [state, setState] = useState(init)
//            ^^^^^^^^
```

- The second element of the arrays is the *setter* function
- You call this function to change the current state

You must always use the setter to update the value - Why?

- so that react knows it needs to *re-render* the component

Example:

Let's say we set up the state like this:

```
const [name, setName] = useState("Jake")
```

Then in a callback function, we could have:

```
setName("Nikela")
```

This would change `name` to be `"Nikela"`, rather than `"Jake"`

The Counter Example

```
function Counter() {
  const [count, setCount] = useState(0)

  return <div className="flex flex-row w-full justify-between">
    <button onClick={() => setCount(count - 1)}>-</button>
    <span>{count}</span>
    <button onClick={() => setCount(count + 1)}>+</button>
  </div>
}
```

When do components render?

- Once at the start (called 'mounting')
- Whenever a parent re-renders
- Whenever state changes

Problem: Recursive Re-rendering

Code that you leave at the top level of your component will run every time the component renders:

```
function MyComponent() {
  const res = foo()
  //          ^ will be called with every render
  return <div></div>

}
```

Problem: Recursive Re-rendering

Let's imagine that we want to fetch some data from an API:

```
function MyComponent() {
  const [data, setData] = useState("Loading awful joke...")


  fetch("https://icanhazdadjoke.com/", { headers: { Accept: "application/json" } })
      .then(data => data.json())
      .then(data => setData(data.joke))

  return <div>{data}</div>

}
```

What's the problem with this?

Problem: Recursive Re-rendering

- Render calls `fetch`
- `fetch` completes and updates `data`
- this is a state change, which triggers a re-render ...
- This code will cause an infinite loop of rendering!
- Solution: `useEffect`

Anatomy of a `useEffect` call

```
    useEffect(setup, deps)
//  ^^^^^^^^^
```

- `useEffect` let's us attach arbitrary behaviour to the component lifecycle

Anatomy of a `useEffect` call

```
    useEffect(setup, deps)
//            ^^^^^^^^^^^^
```

- It takes two arguments

Anatomy of a `useEffect` call

```
    useEffect(setup, deps)
//            ^^^^^
```

- The first is the 'effect' - the actual code that runs
- This should be a function

Anatomy of a `useEffect` call

```
    useEffect(() => console.log("hello"), deps)
//            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

- For example, this will just print "hello" to the browser console

Anatomy of a `useEffect` call

```
   useEffect(setup, deps)
//                  ^^^^
```

- The second argument is called the dependency array
- This controls when the effect runs

Anatomy of a `useEffect` call

```
   useEffect(setup, )
//                  ^
```

- No dependencies: effect runs on every component re-render

Anatomy of a `useEffect` call

```
   useEffect(setup, [x, y, z])
//                  ^^^^^^^^^
```

- If we pass an array, the effect runs whenever one of the elements of the list changes
- So here, whenever `x`, `y`, or `z` changes

Anatomy of a `useEffect` call

```
   useEffect(setup, [])
//                  ^^
```

- Special case: if we pass an empty list, the effect runs on mount ...
- and then never again - since none of the elements in this array will ever change

We can use this to implement our data fetching properly:

```
function MyComponent() {
  const [data, setData] = useState("Loading awful joke...")

  useEffect(()=> {
    fetch("https://icanhazdadjoke.com/", { headers: { Accept: "application/json" } })
      .then(data => data.json())
      .then(data => setData(data.joke))
  }, [])

  return <div>{data}</div>

}
```

Effects can be pretty tricky

Some people call it `useFootGun` because it's so prone to error

Just using `useEffect` does not automatically fix all your problems

For example ...

It's possible to re-create the recursive re-render problem with a `useEffect` call:

```
function MyComponent() {
  const [data, setData] = useState("Loading awful joke...")

  useEffect(()=> {
    fetch("https://icanhazdadjoke.com/", { headers: { Accept: "application/json" } })
      .then(data => data.json())
      .then(data => setData(data.joke))
  }, [data])

  return <div>{data}</div>

}
```

Why does this cause an issue?

React with Django

React with Django

Most of this course has been taught on django

Django is a nice backend framework, react is much better for frontend

Roughly two main ways to join them up:

1. "Islands"
2. Django backend/api, react frontend

Islands

Compile the react component, serve it out as a bundle via django

So named since you get "islands" of interactivity in your frontend

- I have done this
- I don't recommend it
  - It's quite fiddly, very ugly
  - I'd suggest picking a frontend solution and sticking with it

Django backend, react frontend

Develop two applications:

- the django part
  - concerned only with the data processing
- the react part
  - concerned only with presentation and interactivity
- react part should fetch data from the backend, similar to how I showed earlier with `useEffect`

This is a much better model, if you want to use both at once

Advantages of React:

- Natural, composable way to build UIs
    - Components are perhaps *the best model* that we have come up with so far
- Makes programming client-side interactivity much easier
- Provides a structured way to approach problem
- Big ecosystem - if you have a problem, someone has probably already solved it
    - And published it as a package on npm

Disadvantages of React: not actually javascript

- JSX is not actually javascript - you need a build system
  - Javascript build systems are hell - avoid setting one up whenever you can
- Next and Vite are good mitigations - they handle the painful bit for you
- But some solutions avoid the issue entirely, and are easier to get working quickly

Disadvantages of React: React can be quite heavy

- in terms of bundle size …
- and mental load
- Some people violently advocate for simpler and less bloated solutions
- Some alternatives try to mitigate the complexity of react:
  - htmx made a big splash in the web-js discussion
- But it's horses for courses; sometimes heavy is what you need

That's all folks!

Questions?

Or

Email me: `J.Trevor.1@research.gla.ac.uk`